

# **Competitive Programming**

## **Section 8: Binary search on the answer**

Competitive Programming Course

2026-07-16

# Table of contents

|    |                                         |    |
|----|-----------------------------------------|----|
| 1  | Why binary search on the answer matters | 3  |
| 2  | Monotone predicates                     | 4  |
| 3  | Minimum feasible template               | 5  |
| 4  | Maximum feasible template               | 6  |
| 5  | Bounds matter                           | 7  |
| 6  | Feasibility checks                      | 8  |
| 7  | Worked example: maximum distance        | 9  |
| 8  | Worked example: minimum capacity        | 10 |
| 9  | Overflow safety                         | 11 |
| 10 | Common mistakes                         | 12 |
| 11 | Before you submit                       | 13 |
| 12 | Exercises                               | 14 |

# 1 Why binary search on the answer matters

Sometimes the answer is hard to construct directly, but easy to check.

Examples:

- Can we place  $k$  objects with distance at least  $d$ ?
- Can we ship all packages with capacity  $C$ ?
- Is there an integer  $x$  with  $x*x \geq n$ ?

If the yes/no answer changes monotonically as the candidate answer changes, we can binary search the answer.

The pattern is:

```
candidate answer
  |
  v
feasibility predicate: true or false
  |
  v
binary search over candidate values
```

## 2 Monotone predicates

A predicate is **monotone** if its truth values change direction at most once.

For minimum feasible searches:

```
candidate:  0  1  2  3  4  5  6
predicate:  F  F  F  T  T  T  T
```

We want the first T.

For maximum feasible searches:

```
candidate:  0  1  2  3  4  5  6
predicate:  T  T  T  T  F  F  F
```

We want the last T.

Before writing binary search, write the predicate direction in words.

### 3 Minimum feasible template

Use this when larger candidates remain feasible after the first feasible point.

Example:

smallest  $x$  such that  $x*x \geq n$

If  $x$  works, every larger  $x$  also works.

Template:

```
lo = known lower bound
hi = known feasible upper bound
while lo < hi:
    mid = (lo + hi) // 2
    if feasible(mid):
        hi = mid
    else:
        lo = mid + 1
answer = lo
```

Invariant:

```
answer is always in [lo, hi]
hi is feasible
```

## 4 Maximum feasible template

Use this when smaller candidates remain feasible after the last feasible point.

Example: imagine allowed positions 1, 2, 4, 8, 9 and three routers to place. We must choose three of those positions. The **minimum distance** of a choice is the smallest gap between two consecutive chosen routers. For example, choosing positions 1, 4, 8 gives gaps 3 and 4, so its minimum distance is 3.

The goal is to make that smallest gap as large as possible. For a candidate  $d$ , define:

```
feasible(d) = can we choose at least k positions so that every consecutive
               pair we choose is at distance at least d?
```

For the positions above, `feasible(3)` is true because we can choose 1, 4, 8. But `feasible(4)` is false: after choosing one allowed position, there is no way to choose two more while keeping every gap at least four. Therefore the best answer is 3.

If distance  $d$  works, every smaller distance also works: relaxing a required gap can never make an existing placement invalid. This gives the pattern `True, True, ..., False, False`, so we search for the last `True`.

Template with an exclusive bad upper bound:

```
lo = known feasible value
hi = known infeasible value
while lo + 1 < hi:
    mid = (lo + hi) // 2
    if feasible(mid):
        lo = mid
    else:
        hi = mid
answer = lo
```

Invariant:

```
lo is feasible
hi is infeasible
```

## 5 Bounds matter

Binary search needs correct initial bounds.

For square threshold:

```
0 <= n <= 1018  
answer <= 109
```

because:

```
(109)2 = 1018
```

For shipping capacity:

```
lower bound = max package weight  
upper bound = sum of all package weights
```

The lower bound is necessary because every package must fit alone. The upper bound is feasible because one day could ship everything.

## 6 Feasibility checks

Binary search is only as good as its predicate.

For maximum minimum distance:

```
can_place(d):  
    choose the leftmost position  
    repeatedly choose the next position at least d away  
    return whether at least k positions were chosen
```

This greedy check works because choosing earlier never hurts future placement.

For minimum shipping capacity, the input is a sequence of package weights and a number of available days. Packages must be shipped in their given order; each day ships one consecutive block of not-yet-shipped packages; a package cannot be split. A candidate capacity  $C$  is feasible if all packages can be shipped in **at most** the allowed number of days.

For example, with weights 1, 2, 3, 4, 5 and two days, capacity  $C = 9$  is feasible because we can ship 1, 2, 3 on day one and 4, 5 on day two.

The feasibility check is:

```
days_needed(C):  
    scan packages in order  
    start a new day when the next package would exceed C
```

This greedy check uses the fewest days for that capacity because each day is filled as much as possible before moving on.



## 7 Worked example: maximum distance

Positions:

```
1 2 4 8 9
```

Need to choose 3.

Check  $d = 3$ :

```
choose 1
next at least 4 -> choose 4
next at least 7 -> choose 8
success
```

Check  $d = 4$ :

```
choose 1
next at least 5 -> choose 8
next at least 12 -> none
failure
```

So the answer is 3.

## 8 Worked example: minimum capacity

Weights:

```
1 2 3 4 5
```

Days:

```
2
```

Capacity 8:

```
day 1: 1 2 3 = 6  
day 2: 4 = 4  
day 3: 5 = 5  
needs 3 days -> fail
```

Capacity 9:

```
day 1: 1 2 3 = 6  
day 2: 4 5 = 9  
needs 2 days -> success
```

The minimum feasible capacity is 9.

## 9 Overflow safety

Binary search often computes midpoints and products near constraints.

Use:

```
mid = lo + (hi - lo) / 2;
```

instead of:

```
mid = (lo + hi) / 2;
```

For  $\text{mid} * \text{mid}$  near  $10^{18}$ , use `__int128` in C++:

```
(__int128)mid * mid
```

Python integers do not overflow, but the logic can still be off by one.

## 10 Common mistakes

- Binary searching when the predicate is not monotone.
- Using a minimum-feasible template for a maximum-feasible problem.
- Choosing an upper bound that is not actually feasible.
- Forgetting that `hi` may be exclusive in some templates.
- Infinite loops caused by `mid = (lo + hi) / 2` and wrong updates.
- Overflowing `mid * mid`.
- Writing a feasibility check that is slower than expected.

# 11 Before you submit

Write down:

1. What is the candidate answer?
2. Is the predicate **False...True** or **True...False**?
3. What are the initial bounds?
4. Which bound is guaranteed feasible?
5. What is the complexity of one predicate check?
6. What is the total complexity after multiplying by `log range`?

## 12 Exercises

The local exercises cover:

1. `a_square_threshold`: first true over integers.
2. `b_max_min_distance`: last true with a greedy placement check.
3. `c_minimum_capacity`: first true with a greedy scheduling check.

Run:

```
python3 sections/08_binary_search_answer/check.py
```